

K P Nidhi
Reg No: 261100690024

Importance of Cryptography: HTTP vs HTTPS Credential Transmission

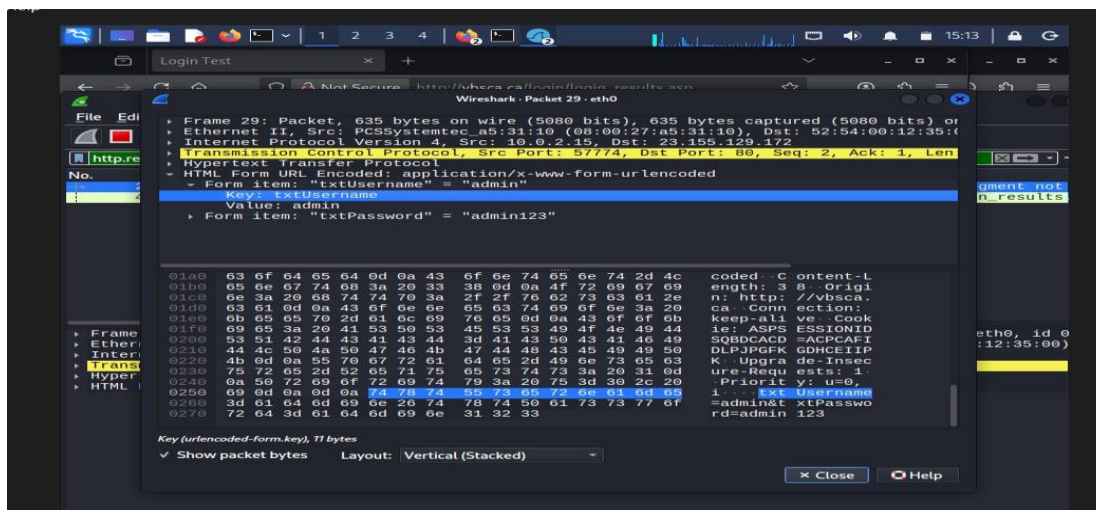
Objective

The goal of this practical was to see, firsthand, how login credentials actually travel over a network when a site uses HTTP versus when it uses HTTPS. Wireshark was used to capture traffic and compare an unencrypted HTTP connection with an encrypted HTTPS one.

Procedure

For this experiment, dummy login credentials were entered on an HTTP-based test login page and an HTTPS-secured page. At the same time, Wireshark was used to capture the network traffic through the eth0 interface. The captured packets were then examined to see whether the username and password could be viewed directly.

1.HTTP Traffic Analysis



Observation:

Packet 29 shows an HTTP POST request being sent to vbsca.ca/login/login_results.asp through port 80. Once the packet was opened up in Wireshark, the submitted form data was sitting right there in plain view.

The captured information included:

- txtUsername = admin
- txtPassword = admin123

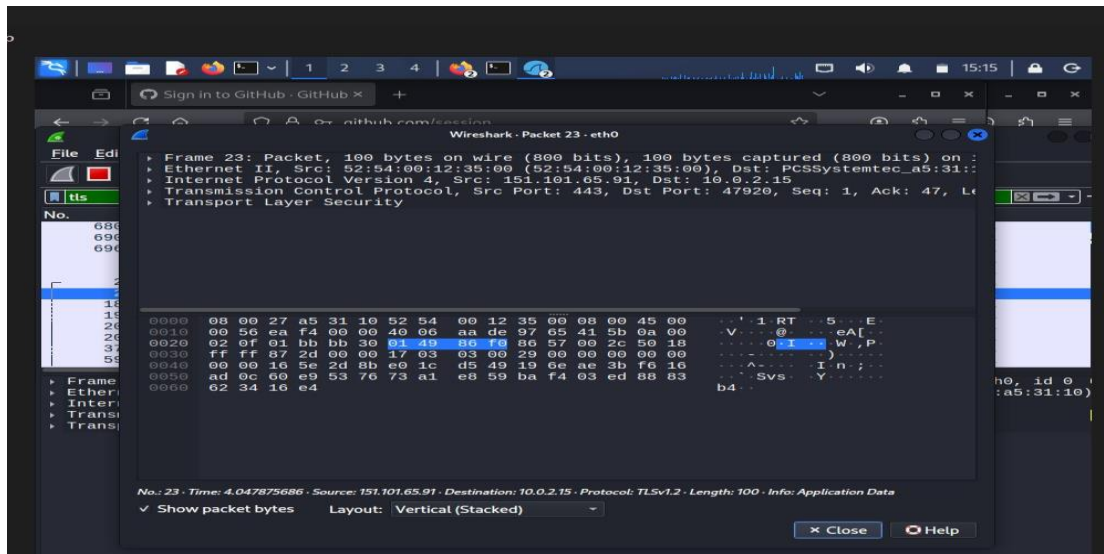
The same information could also be read straight off the packet's raw data section:

txtUsername=admin&txtPassword=admin123

Explanation: This basically confirms that HTTP does not encrypt anything it sends. Because the username and password go out as plain text, anyone capturing traffic on the same network could grab the credentials without needing to do any extra work.

This is exactly why using HTTP for login pages is risky, especially when something sensitive like a password or personal detail is being sent.

2.HTTPS Traffic Analysis



Observation:

Packet 23 shows TLS v1.2 Application Data being transmitted through port 443. Unlike the HTTP packet, the username and password cannot be seen directly in Wireshark.

The packet contains encrypted data that appears as unreadable characters in the hex/ASCII section. Information such as the source IP, destination IP, protocol, and packet length can still be observed, but the actual contents of the communication are protected.

Explanation:

This happens because HTTPS uses TLS encryption to protect the data before it is sent across the network. Even if someone captures the packets using a tool such as Wireshark, they cannot simply read the username and password from the captured traffic.

Therefore, HTTPS provides much better protection for sensitive information during transmission.

3.Comparison

Aspect	HTTP	HTTPS
Data format	Plaintext	Encrypted using TLS
Port	80	443
Risk of credential interception	High	Much lower
Confidentiality	Not provided	Provided through encryption

Integrity	Not provided by HTTP itself	Protected by TLS
-----------	-----------------------------	------------------

4.Role of Encryption

This experiment really drove home why encryption matters when sensitive information is sent over a network.

With HTTP, login credentials go out as plain text, so anyone monitoring the traffic could potentially read the username and password without even needing to decrypt anything.

HTTPS solves this problem by using TLS encryption. The data is encrypted before it travels across the network, so even if someone captures the packets, the actual contents cannot be easily understood.

TLS also provides other important security benefits, such as integrity, which helps detect whether data has been modified during transmission, and authentication, which helps verify that the user is communicating with the intended server.

5.Conclusion

Overall, this Wireshark exercise made the difference between HTTP and HTTPS pretty obvious. In the HTTP capture, the dummy username and password were sitting there in plain text, which shows just how easily credentials can be exposed when there's no encryption involved.

In the HTTPS capture, the same kind of sensitive information was protected by TLS and simply could not be read directly from the captured packets.

So in short, cryptography plays a genuinely important role in keeping sensitive information safe while it's being transmitted over a network. HTTPS should be used instead of HTTP whenever login credentials, personal information, payment details, or any other sensitive data are involved.